Note on the Model Implementation

1. Swaption Pricing and Calibration Pipeline

This section documents the implementation pipeline for European swaption pricing under the G2++ model, starting from the affine bond-basket coefficients and ending with the calibration objective. The objective is to make explicit the correspondence between the mathematical formula and the MATLAB methods used in the class `GaussianTwoFactors`.

1.1. Notation and model objects

Under the G2++ model, the short rate is

$$r_t = x_t + y_t + \varphi(t),$$

where

$$dx_t = -ax_t\,dt + \sigma(t)\,dW_t^x, \qquad dy_t = -by_t\,dt + \eta(t)\,dW_t^y, \qquad d\langle W^x, W^y\rangle_t = \rho\,dt.$$

At the swaption expiry T, the zero-coupon bond price admits the affine representation

$$P(T,t_i) = A(T,t_i)\exp\left(-B(a,T,t_i)x_T - B(b,T,t_i)y_T\right),$$

where

$$B(\alpha,T,t_i) = \frac{1-e^{-\alpha(t_i-T)}}{\alpha}.$$

For a payer swaption, the payoff at expiry T can be written as

$$\left[1 - \sum_{i=1}^{n} c_i P(T,t_i)\right]^+,$$

with coefficients

$$c_i = K\delta_i, \quad i = 1,\dots,n-1, \qquad c_n = 1 + K\delta_n.$$

The receiver payoff is the opposite side of the same basket:

$$\left[\sum_{i=1}^{n} c_i P(T,t_i) - 1\right]^+.$$

We use the convention

$$\omega = \begin{cases} 1, & \text{payer swaption,} \\ -1, & \text{receiver swaption.} \end{cases}$$

1.2. `getLambdaAndBy`: bond-basket coefficients

Mathematical responsibility. For fixed x, define

$$\lambda_i(x) = c_i A(T,t_i)\exp\left(-B(a,T,t_i)x\right).$$

Then the bond basket becomes

$$\sum_{i=1}^{n} c_i P(T,t_i) = \sum_{i=1}^{n} \lambda_i(x)\exp\left(-B(b,T,t_i)y\right).$$

The method `getLambdaAndBy` returns the vector

$$\lambda(x) = (\lambda_1(x), \ldots, \lambda_n(x))^\top$$

and the vector

$$B^y = (B(b, T, t_1), \ldots, B(b, T, t_n))^\top .$$

Software responsibility. The method combines the standardized swaption instrument with the affine bond-basket object.

```
[lambda, By] = getLambdaAndBy(obj, strike, currMaturity, ...
                              ulSwapSteps, x_state, args)
```

Its implementation depends on

`getSwaptionInstrument` and `getSwaptionBondBasket`.

The numerical computation is

$$\lambda_i = c_i A_i e^{-B_i^x x}.$$

```
coeffs = instr.coeffs(:);
A0Ti   = basket.A0Ti(:);
Bx0Ti  = basket.Bx0Ti(:);
By0Ti  = basket.By0Ti(:);

lambda = coeffs .* A0Ti .* exp(-Bx0Ti .* x_state);
By     = By0Ti;
```

Implementation trap. The scalar state variable must be used consistently. If an intermediate variable `x` is created from `x_state`, the final formula must use `x`, not the original potentially non-scalar input.

1.3. `getYBarValue`: exercise-boundary equation

Mathematical responsibility. For each fixed x, the exercise boundary $\bar{y}(x)$ is the unique solution of

$$1 = \sum_{i=1}^{n} \lambda_i(x) \exp\left(-B_i^y \bar{y}(x)\right).$$

Equivalently,

$$f_x(y) = 1 - \sum_{i=1}^{n} \lambda_i(x) e^{-B_i^y y} = 0.$$

Software responsibility. The method returns the scalar residual

$$f_x(y).$$

```
value = getYBarValue(obj, y, lambda, By)
```

with implementation

```
value = 1 - sum(lambda .* exp(-By .* y));
```

Numerical property. When $\lambda_i > 0$ and $B_i^y > 0$, the function is strictly increasing:

$$f_x'(y) = \sum_{i=1}^{n} \lambda_i B_i^y e^{-B_i^y y} > 0.$$

Therefore the root is unique.

1.4. `yBarSolverFromLambda`: solving for $\bar{y}(x)$

Mathematical responsibility. The method computes

$$\bar{y}(x) = f_x^{-1}(0),$$

where

$$f_x(y) = 1 - \sum_{i=1}^{n} \lambda_i(x) e^{-B_i^y y}.$$

Software responsibility. The method receives already-computed $\lambda(x)$ and B^y, and solves the scalar equation by a robust bracketed root-finding method.

```
yBar = yBarSolverFromLambda(obj, lambda, By, args)
```

Recommended implementation logic:

1. Build scalar function `scalarF`(y).
2. Start with a small bracket around a seed.
3. Expand the bracket until the signs differ.
4. Call `fzero`.
5. Check the residual.

```
scalarF = @(y) obj.getYBarValue(y, lambda, By);

yLow  = ySeed - 0.01;
yHigh = ySeed + 0.01;

while scalarF(yLow) * scalarF(yHigh) > 0
    width = yHigh - yLow;
    yLow  = yLow  - 2 * width;
    yHigh = yHigh + 2 * width;
end

yBar = fzero(scalarF, [yLow, yHigh]);
```

Implementation trap. The fallback solver must not use an undefined function such as `scalarF` before it is created. Also, `fzero` should be used with a bracket, not only with a single starting value.

1.5. Gaussian conditioning at expiry

Under the T-forward measure, the state vector at expiry satisfies

$$(X, Y) := (x_T, y_T) \sim \mathcal{N}\left(\begin{pmatrix}\mu_x \\ \mu_y\end{pmatrix}, \begin{pmatrix}\sigma_x^2 & \rho_{xy}\sigma_x\sigma_y \\ \rho_{xy}\sigma_x\sigma_y & \sigma_y^2\end{pmatrix}\right).$$

Conditioning on $X = x$,

$$Y \mid X = x \sim \mathcal{N}\left(\mu_{Y|x}, \sigma_{Y|x}^2\right),$$

where

$$\mu_{Y|x} = \mu_y + \rho_{xy}\frac{\sigma_y}{\sigma_x}(x - \mu_x),$$

and

$$\sigma_{Y|x}^2 = \sigma_y^2(1 - \rho_{xy}^2).$$

Equivalently, using covariance notation,

$$\mu_{Y|x} = \mu_y + \frac{\mathrm{Cov}(X, Y)}{\mathrm{Var}(X)}(x - \mu_x),$$

and

$$\sigma_{Y|x}^2 = \mathrm{Var}(Y) - \frac{\mathrm{Cov}(X,Y)^2}{\mathrm{Var}(X)}.$$

The implementation should use the covariance form, since the class naturally computes

$$\mathrm{Var}(X), \qquad \mathrm{Var}(Y), \qquad \mathrm{Cov}(X,Y).$$

1.6. `h1Function`: standardized exercise boundary

Mathematical responsibility. Define

$$h_1(x) = \frac{\bar{y}(x) - \mu_{Y|x}}{\sigma_{Y|x}}.$$

Using the explicit conditional moments,

$$h_1(x) = \frac{\bar{y}(x) - \mu_y}{\sigma_y\sqrt{1-\rho_{xy}^2}} - \frac{\rho_{xy}(x-\mu_x)}{\sigma_x\sqrt{1-\rho_{xy}^2}}.$$

Software responsibility. The numerically simplest implementation receives the conditional mean and conditional standard deviation:

```
h1 = h1Function(obj, yBar, muYCond, sigmaYCond)
```

and computes

$$h_1 = \frac{\bar{y} - \mu_{Y|x}}{\sigma_{Y|x}}.$$

```
h1 = (yBar - muYCond) ./ sigmaYCond;
```

This is equivalent to the explicit formula above.

1.7. `h2Function`: exponential-tilted boundary

Mathematical responsibility. For each bond maturity t_i, define

$$h_{2,i}(x) = h_1(x) + B(b,T,t_i)\sigma_{Y|x}.$$

Since

$$\sigma_{Y|x} = \sigma_y\sqrt{1-\rho_{xy}^2},$$

this is equivalent to

$$h_{2,i}(x) = h_1(x) + B(b,T,t_i)\sigma_y\sqrt{1-\rho_{xy}^2}.$$

Software responsibility. The method returns a vector of length n:

```
h2 = h2Function(obj, h1, By, sigmaYCond)
```

with implementation

```
h2 = h1 + By(:) .* sigmaYCond;
```

1.8. `kappaFunction`: Gaussian exponential adjustment

Mathematical responsibility. For each i,

$$\kappa_i(x) = -B_i^y \left[\mu_{Y|x} - \frac{1}{2} B_i^y \sigma_{Y|x}^2 \right].$$

Equivalently,

$$\kappa_i(x) = -B_i^y \mu_{Y|x} + \frac{1}{2} (B_i^y)^2 \sigma_{Y|x}^2.$$

This adjustment appears because

$$\mathbb{E}\left[e^{-B_i^y Y} \mathbf{1}_{\{Y > \bar{y}\}} \,\middle|\, X = x \right]$$

is computed under a Gaussian density and requires an exponential completion of the square.

Software responsibility. The method returns the vector

$$\kappa(x) = (\kappa_1(x), \ldots, \kappa_n(x))^\top.$$

```
kappa = kappaFunction(obj, By, muYCond, varYCond)
```

with implementation

```
kappa = -By .* muYCond + 0.5 .* (By.^2) .* varYCond;
```

Implementation trap. The object $\kappa_i(x)$ is not the same as the exercise boundary $\bar{y}(x)$. They must be represented by different variables.

1.9. `brigoSwaptionIntegrand`: one-dimensional integrand

Mathematical responsibility. The swaption price can be written as a one-dimensional integral over x:

$$\mathrm{Swpt}_0 = NP(0,T) \int_{-\infty}^{+\infty} f_X(x)\, \omega \left[\Phi(-\omega h_1(x)) - \sum_{i=1}^{n} \lambda_i(x) e^{\kappa_i(x)} \Phi(-\omega h_{2,i}(x)) \right] dx.$$

For numerical stability, integrate over the standardized variable

$$z = \frac{x - \mu_x}{\sigma_x}, \qquad x = \mu_x + \sigma_x z, \qquad z \sim \mathcal{N}(0,1).$$

Then

$$dx\, f_X(x) = \phi(z)\, dz,$$

and the integrand becomes

$$\omega \left[\Phi(-\omega h_1(z)) - \sum_{i=1}^{n} \lambda_i(z) e^{\kappa_i(z)} \Phi(-\omega h_{2,i}(z)) \right] \phi(z).$$

Software responsibility. The method computes the integrand value for one or several values of z.

```
output = brigoSwaptionIntegrand(obj, z, omega, coeffs, A0Ti, ...
                                Bx, By, gaussianState)
```

The implementation sequence is:

1. Convert z into $x = \mu_x + \sigma_x z$.
2. Compute $\lambda_i(x)$.
3. Solve $\bar{y}(x)$.
4. Compute $\mu_{Y|x}$ and $\sigma_{Y|x}^2$.
5. Compute $h_1(x)$, $h_2(x)$, and $\kappa_i(x)$.

6. Evaluate the Brigo bracket.
7. Multiply by $\phi(z)$.

The core code block is

```
xVal = muX + sigmaX * z;

lambda = coeffs .* A0Ti .* exp(-Bx .* xVal);

yBar = obj.yBarSolverFromLambda(lambda, By);

muYCond = muY + (covXY / varX) * (xVal - muX);
varYCond = varY - covXY^2 / varX;
sigmaYCond = sqrt(max(varYCond, 0));

h1 = obj.h1Function(yBar, muYCond, sigmaYCond);
h2 = obj.h2Function(h1, By, sigmaYCond);
kappa = obj.kappaFunction(By, muYCond, varYCond);

bracket = normcdf(-omega .* h1) ...
        - sum(lambda .* exp(kappa) .* normcdf(-omega .* h2));

output = omega .* bracket .* normpdf(z);
```

Implementation trap. The sign of h_1 and the sign inside the normal CDF must be consistent. With

$$h_1 = \frac{\bar{y} - \mu_{Y|x}}{\sigma_{Y|x}},$$

the formula uses

$$\Phi(-\omega h_1).$$

Changing the definition of h_1 requires changing the CDF signs.

1.10. `priceSwaptionG2PP`: orchestration method

Mathematical responsibility. The method computes

$$\mathrm{Swpt}_0 = NP(0,T) \int_{-\infty}^{+\infty} \texttt{brigoSwaptionIntegrand}(z)\, dz.$$

The discounting numeraire is $P(0,T)$, where T is the option expiry.

Software responsibility. The method orchestrates the full pricing pipeline:

1. Build the standardized swaption instrument.
2. Build the affine bond basket.
3. Compute the Gaussian law of (x_T, y_T) under the T-forward measure.
4. Set $\omega = 1$ for payer and $\omega = -1$ for receiver.
5. Integrate the Brigo integrand over a finite standard-normal range, for example $[-8, 8]$.
6. Multiply by notional and $P(0,T)$.

```
price = priceSwaptionG2PP(obj, strike, currMaturity, ulSwapSteps, args)
```

Core implementation:

```
instr = obj.getSwaptionInstrument(strike, currMaturity, ulSwapSteps, ...);
basket = obj.getSwaptionBondBasket(instr.expiry, instr.paymentDates);
gaussianState = obj.getStateGaussianAtExpiry(instr.expiry);

integrand = @(z) obj.brigoSwaptionIntegrand( ...
    z, omega, instr.coeffs(:), basket.A0Ti(:), ...
```

```
    basket.Bx0Ti(:), basket.By0Ti(:), gaussianState);

expectation = integral(integrand, -8, 8, ...
    'ArrayValued', true, ...
    'AbsTol', absTol, ...
    'RelTol', relTol);

price = instr.notional * basket.P0T0 * expectation;
```

Validation checks. The method should satisfy:

$$\text{Price}_{payer} \geq 0, \qquad \text{Price}_{receiver} \geq 0.$$

It should also satisfy payer–receiver parity:

$$\text{Payer} - \text{Receiver} = N\left[P(0,T) - \sum_{i=1}^{n} c_i P(0,t_i)\right].$$

2. Calibration Pipeline

The calibration layer wraps the pricing method into an optimizer-friendly objective. The pricing method remains responsible only for pricing a single instrument. Calibration is handled outside the core pricing formula.

2.1. Calibration instruments

A calibration set should be represented as a flat array of structures:

```
calibSet(k).expiry
calibSet(k).tenor
calibSet(k).paymentDates
calibSet(k).quote
calibSet(k).quoteType
calibSet(k).direction
calibSet(k).notional
calibSet(k).strikeConvention
calibSet(k).fixedStrike
calibSet(k).weight
```

This avoids coupling the optimizer to the shape of a market matrix.

2.2. `getSwaptionForwardAndAnnuityG2PP`

Mathematical responsibility. Given expiry T and payment dates $t_1, \ldots, t_n$, the single-curve annuity is

$$A_0 = \sum_{i=1}^{n} \delta_i P(0,t_i),$$

and the forward swap rate is

$$S_0 = \frac{P(0,T) - P(0,t_n)}{A_0}.$$

Software responsibility. The method computes the market forward swap rate and annuity used for ATM strike construction and quote-to-price conversion.

```
[F, annuity, delta, P0T0, P0Ti] = ...
    getSwaptionForwardAndAnnuityG2PP(model, expiry, paymentDates)
```

2.3. Quote-to-price conversion

Market data may be provided as prices, Black volatilities, or normal volatilities. The calibration objective should compare model prices with market prices or normalized market prices.

For a normal volatility σ_N, the Bachelier swaption price is

$$V_N = NA_0 \left[\omega(S_0 - K)\Phi(\omega d) + \sigma_N \sqrt{T}\phi(d)\right],$$

where

$$d = \frac{S_0 - K}{\sigma_N \sqrt{T}}.$$

For a Black volatility σ_B, assuming $S_0 > 0$ and $K > 0$,

$$V_B = NA_0\omega \left[S_0\Phi(\omega d_1) - K\Phi(\omega d_2)\right],$$

with

$$d_1 = \frac{\log(S_0/K) + \frac{1}{2}\sigma_B^2 T}{\sigma_B\sqrt{T}}, \qquad d_2 = d_1 - \sigma_B\sqrt{T}.$$

Software responsibility. The helper

```
marketPrice = swaptionMarketPriceFromQuote(model, instr)
```

resolves

$$\texttt{quoteType} \in \{\texttt{price}, \texttt{normal}, \texttt{black}\}$$

and returns a market price compatible with `priceSwaptionG2PP`.

2.4. Parameter transformation

Mathematical responsibility. The G2++ admissibility constraints are

$$a > 0, \qquad b > 0, \qquad \sigma > 0, \qquad \eta > 0, \qquad -1 < \rho < 1.$$

The optimizer should therefore work with unconstrained variables

$$\theta_{\text{raw}} = (\alpha, \beta, u, v, z),$$

mapped by

$$a = e^{\alpha}, \qquad b = e^{\beta}, \qquad \sigma = e^{u}, \qquad \eta = e^{v}, \qquad \rho = \tanh z.$$

Software responsibility. The inverse transformation is

```
thetaRaw = g2ppInverseTransformBrigo5(a, b, sigma, eta, rho)
```

and the application to the model is

```
params = g2ppApplyTransformBrigo5(model, thetaRaw)
```

For a constant-volatility calibration, the implementation assigns

$$\sigma_i = \sigma, \qquad \eta_i = \eta$$

for every volatility bucket.

```
model.meanReversion_x = exp(thetaRaw(1));
model.meanReversion_y = exp(thetaRaw(2));
model.volatility_sigma = sigma * ones(size(model.volatility_sigma));
model.volatility_eta   = eta   * ones(size(model.volatility_eta));
model.correlation      = tanh(thetaRaw(5));
```

2.5. Calibration residuals

Mathematical responsibility. For each calibration instrument k, define

$$\varepsilon_k(\theta) = V_k^{model}(\theta) - V_k^{mkt}.$$

For numerical stability across maturities and notionals, use normalized price residuals:

$$\varepsilon_k^{norm}(\theta) = \frac{V_k^{model}(\theta) - V_k^{mkt}}{N_k A_k}.$$

With weights w_k, the least-squares objective is

$$\min_\theta \sum_{k=1}^{m} w_k \left(\varepsilon_k^{norm}(\theta)\right)^2.$$

Software responsibility. The residual method performs the following operations:

1. Apply transformed parameters to the model.
2. Loop through calibration instruments.
3. Resolve strike K_k.
4. Compute market price V_k^{mkt}.
5. Compute model price V_k^{model}.
6. Return residual vector and diagnostics.

```
[residuals, diagnostics] = ...
    g2ppCalibrationResiduals(thetaRaw, model, calibSet, args)
```

Core implementation:

```
params = g2ppApplyTransformBrigo5(model, thetaRaw);

for k = 1:numel(calibSet)
    instr = calibSet(k);

    K = getCalibrationStrikeG2PP(model, instr);

    [F, A] = getSwaptionForwardAndAnnuityG2PP( ...
        model, instr.expiry, instr.paymentDates);

    marketPrice = swaptionMarketPriceFromQuote(model, instr);

    modelPrice = model.priceSwaptionG2PP( ...
        K, instr.expiry, instr.paymentDates, ...
        'direction', instr.direction, ...
        'notional', instr.notional);

    residuals(k) = sqrt(instr.weight) ...
        * (modelPrice - marketPrice) ...
        / max(instr.notional * A, 1e-12);
end
```

Robustness rule. The residual function must never crash inside the optimizer. If a pricing call fails, it should return a large finite penalty.

2.6. Calibration driver

Mathematical responsibility. The calibration driver solves

$$\theta^\star = \arg\min_\theta \|R(\theta)\|^2,$$

where

$$R(\theta) = (\varepsilon_1(\theta), \ldots, \varepsilon_m(\theta))^\top.$$

Software responsibility. The driver constructs the residual function and calls a nonlinear least-squares optimizer.

```
result = calibrateG2PPToSwaptionSet(model, calibSet, theta0Raw, args)
```

Recommended solver:

```
lsqnonlin(residualFun, theta0Raw, [], [], opts)
```

The output should contain

```
result.thetaRaw
result.params
result.resnorm
result.finalResiduals
result.diagnostics
result.rmse
result.maxAbsError
```

where

$$\mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_{k=1}^{m}\varepsilon_k^2}.$$

2.7. Calibration validation

The minimum validation checks are:

1. **Finite residuals at initial parameters.**
$$R(\theta_0) \in \mathbb{R}^m.$$
2. **Admissible calibrated parameters.**
$$a, b, \sigma, \eta > 0, \qquad |\rho| < 1.$$
3. **Payer–receiver parity for calibrated prices.**
$$\mathrm{Payer} - \mathrm{Receiver} = N\left[P(0,T) - \sum_i c_i P(0,t_i)\right].$$
4. **Synthetic self-calibration.** Generate market prices from the model itself and verify that the calibration recovers a near-zero objective.
5. **Monotonicity in strike.** For payer swaptions,
$$K_1 < K_2 \implies V^{payer}(K_1) \geq V^{payer}(K_2),$$
while for receiver swaptions,
$$K_1 < K_2 \implies V^{receiver}(K_1) \leq V^{receiver}(K_2).$$

2.8. Pipeline summary

The complete pricing pipeline is

$$\boxed{\texttt{getSwaptionInstrument} \to \texttt{getSwaptionBondBasket} \to \texttt{getLambdaAndBy} \to \texttt{yBarSolverFromLambda} \to \texttt{h1Function}, \texttt{h2Func}}$$

The complete calibration pipeline is

$$\boxed{\texttt{calibSet} \to \texttt{parameterTransform} \to \texttt{priceSwaptionG2PP} \to \texttt{marketQuoteToPrice} \to \texttt{residualVector} \to \texttt{lsqnonlin} \to \texttt{dia}}$$